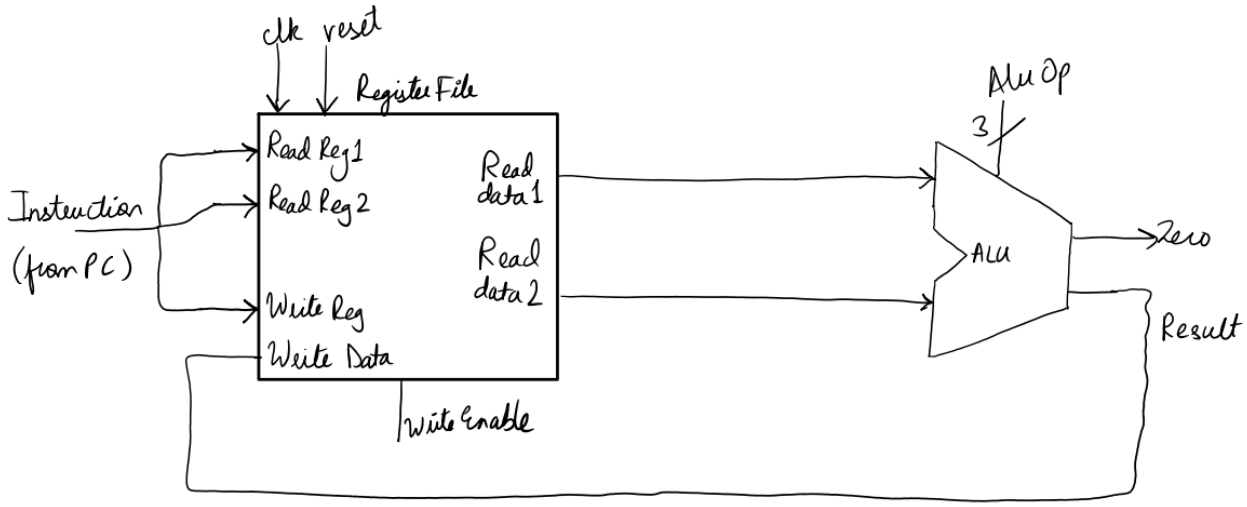
Lab 7: Design and FPGA Implementation of the Register File for a Single-Cycle RISC-V Processor

Objective

Task 1: Register File Block Diagram Design

Block diagram of Register File



Task 2: Verilog Description and Testbench Development

Verilog source file for Register File

```
`timescale 1ns / 1ps
module RegisterFile (
   input  wire clk,
   input  wire rst,
   input  wire WriteEnable,
   input  wire [4:0]  rs1,   // reg 1
   input  wire [4:0]  rs2, // reg 2
   input  wire [4:0]  rd,    // destination reg
   input  wire [31:0] WriteData,
   output wire [31:0] ReadData1, //reg 1 data
   output wire [31:0] ReadData2 //reg 2 data
);

   reg [31:0] regs [31:0]; //32 registers, each 32 bits wide
   integer k; // value 32 for loop
   always @(posedge clk or posedge rst) begin
      if (rst) begin
         //reset clears reg
         for (k = 0; k < 32; k = k + 1)
            regs[k] <= 32'd0;
      end else begin
         if (WriteEnable && (rd != 5'd0))
            regs[rd] <= WriteData;
      end
   end
   assign ReadData1 = (rs1 == 5'd0) ? 32'd0 : regs[rs1];
   assign ReadData2 = (rs2 == 5'd0) ? 32'd0 : regs[rs2];

endmodule
```

Testbench file for Register File

```
`timescale 1ns / 1ps
module RegisterFile_tb;
   reg       clk;
   reg       rst;
   reg       WriteEnable;
   reg  [4:0] rs1, rs2, rd;
   reg  [31:0] WriteData;
   wire [31:0] ReadData1, ReadData2;

   RegisterFile uut (
      .clk(clk),
      .rst(rst),
      .WriteEnable(WriteEnable),
      .rs1(rs1),
      .rs2(rs2),
      .rd(rd),
      .WriteData(WriteData),
      .ReadData1(ReadData1),
      .ReadData2(ReadData2)
   );
```

```
// 100 MHz clock
always #5 clk = ~clk;

task write_reg;
   input [4:0]  addr;
   input [31:0] data;
   begin
      @(negedge clk);          // set up before rising edge
      WriteEnable = 1;
      rd          = addr;
      WriteData   = data;
      @(posedge clk); #1;       // capture on rising edge
      WriteEnable = 0;
   end
endtask

integer pass_count = 0;
integer fail_count = 0;

initial begin
   // --- Initialise ---
   clk        = 0;
   rst        = 1;
   WriteEnable = 0;
   rs1        = 0;
   rs2        = 0;
   rd         = 0;
   WriteData   = 0;

   // Release reset after 3 clock cycles
   repeat (3) @(posedge clk);
   rst = 0;
   #1;
   write_reg(5'd5, 32'hDEAD_BEEF);

   rs1 = 5'd5;
   #1;  // asynchronous read settles immediately
   if (ReadData1 === 32'hDEAD_BEEF) begin
      $display("[PASS] Test i : x5 = 0x%08h (correct)", ReadData1);
      pass_count = pass_count + 1;
   end else begin
      $display("[FAIL] Test i : x5 expected 0xDEADBEEF, got 0x%08h", ReadData1);
      fail_count = fail_count + 1;
   end

   write_reg(5'd0, 32'hFFFF_FFFF);

   rs1 = 5'd0;
   #1;
   if (ReadData1 === 32'd0) begin
      $display("[PASS] Test ii: x0 = 0x%08h (correctly stays zero)", ReadData1);
      pass_count = pass_count + 1;
   end else begin
      $display("[FAIL] Test ii: x0 expected 0x00000000, got 0x%08h", ReadData1);
      fail_count = fail_count + 1;
   end
```

```
    write_reg(5'd10, 32'hCAFE_BABE);

    rs1 = 5'd5;   // should return 0xDEADBEEF
    rs2 = 5'd10;  // should return 0xCAFEBABE
    #1;
    if (ReadData1 === 32'hDEAD_BEEF && ReadData2 === 32'hCAFE_BABE) begin
       $display("[PASS] Test iii: Dual read OK  rs1=0x%08h  rs2=0x%08h", ReadData1, ReadData2);
       pass_count = pass_count + 1;
    end else begin
       $display("[FAIL] Test iii: rs1=0x%08h (exp 0xDEADBEEF)  rs2=0x%08h (exp 0xCAFEBABE)",
              ReadData1, ReadData2);
       fail_count = fail_count + 1;
    end

    write_reg(5'd5, 32'h1234_5678);

    rs1 = 5'd5;
    #1;
    if (ReadData1 === 32'h1234_5678) begin
       $display("[PASS] Test iv : x5 overwritten to 0x%08h (correct)", ReadData1);
       pass_count = pass_count + 1;
    end else begin
       $display("[FAIL] Test iv : x5 expected 0x12345678, got 0x%08h", ReadData1);
       fail_count = fail_count + 1;
    end
    rst = 1;
    @(posedge clk); #1;
    rst = 0;

    rs1 = 5'd5;
    rs2 = 5'd10;
    #1;
    if (ReadData1 === 32'd0 && ReadData2 === 32'd0) begin
       $display("[PASS] Test v  : Reset OK â€“ x5=0x%08h, x10=0x%08h", ReadData1, ReadData2);
       pass_count = pass_count + 1;
    end else begin
       $display("[FAIL] Test v  : After reset x5=0x%08h, x10=0x%08h (expected 0)", ReadData1, ReadData2);
       fail_count = fail_count + 1;
    end

    //
    $display("--------------------------------------------------");
    $display("RegisterFile_tb: %0d PASSED, %0d FAILED", pass_count, fail_count);
    $display("--------------------------------------------------");
    $finish;
  end

endmodule
```

Verilog source file for Top-level (top rf alu.v).

```
`timescale 1ns / 1ps


module top_rf_alu (
  input  wire       clk,
  input  wire       pbin,
  input  wire [15:0] physical_sw,
```

```
    output wire [15:0] physical_leds
);

    wire rst_clean;
    debouncer rst_db (
        .clk  (clk),
        .pbin (pbin),
        .pbout(rst_clean)
    );
    wire [4:0]  fsm_rs1, fsm_rs2, fsm_rd;
    wire [31:0] fsm_wd;
    wire        fsm_we;
    wire [3:0]  fsm_alu_ctrl;
    wire [3:0]  fsm_state_out;

    wire [31:0] rf_rd1, rf_rd2;
    wire [31:0] alu_result;
    wire        alu_zero;

    //

    RegisterFile RF (
        .clk        (clk),
        .rst        (rst_clean),
        .WriteEnable(fsm_we),
        .rs1        (fsm_rs1),
        .rs2        (fsm_rs2),
        .rd         (fsm_rd),
        .WriteData  (fsm_wd),
        .ReadData1  (rf_rd1),
        .ReadData2  (rf_rd2)
    );

    alu_32bit ALU (
        .A          (rf_rd1),
        .B          (rf_rd2),
        .ALUControl(fsm_alu_ctrl),
        .ALUResult (alu_result),
        .Zero       (alu_zero)
    );

    rf_alu_fsm FSM (
        .clk        (clk),
        .rst        (rst_clean),
        .alu_result(alu_result),
        .alu_zero  (alu_zero),
        .rf_we      (fsm_we),
        .rf_rs1     (fsm_rs1),
        .rf_rs2     (fsm_rs2),
        .rf_rd      (fsm_rd),
        .rf_wd      (fsm_wd),
        .alu_ctrl  (fsm_alu_ctrl),
        .state_out (fsm_state_out)
    );

    switches led_writer (
        .clk        (clk),
```

```
        .rst       (rst_clean),
        .writeData  ({16'd0, alu_zero, fsm_state_out, alu_result[10:0]}),
        .writeEnable(1'b1),
        .readEnable (1'b0),
        .memAddress (30'd0),
        .readData   (),
        .leds      (physical_leds)
    );

endmodule

module rf_alu_fsm (
    input  wire       clk,
    input  wire       rst,
    input  wire [31:0] alu_result,
    input  wire       alu_zero,
    // Register File control
    output reg        rf_we,
    output reg  [4:0]  rf_rs1,
    output reg  [4:0]  rf_rs2,
    output reg  [4:0]  rf_rd,
    output reg  [31:0] rf_wd,
    // ALU control
    output reg  [3:0]  alu_ctrl,
    // Status (for LED display)
    output reg  [3:0]  state_out
);

    // ALU control codes - match Lab 6 alu_32bit
    localparam [3:0]
        CTRL_AND = 4'b0000,
        CTRL_OR  = 4'b0001,
        CTRL_ADD = 4'b0010,
        CTRL_SUB = 4'b0110,
        CTRL_XOR = 4'b0100,
        CTRL_SLL = 4'b1000,
        CTRL_SRL = 4'b1001;
    localparam [3:0]
        S_INIT0    = 4'd0,   // write x1 = 0x10101010
        S_INIT1    = 4'd1,   // write x2 = 0x01010101
        S_INIT2    = 4'd2,   // write x3 = 5 (shift amount)
        S_READ_ALU = 4'd3,   // present rs1=x1, rs2=x2, wait for ALU to settle
        S_WR_ADD   = 4'd4,   // write ADD result -> x4
        S_WR_SUB   = 4'd5,   // write SUB result -> x5
        S_WR_AND   = 4'd6,   // write AND result -> x6
        S_WR_OR    = 4'd7,   // write OR  result -> x7
        S_WR_XOR   = 4'd8,   // write XOR result -> x8
        S_READ_SLL = 4'd9,   // present rs1=x1, rs2=x3 for shift ops
        S_WR_SLL   = 4'd10,  // write SLL result -> x9
        S_WR_SRL   = 4'd11,  // write SRL result -> x10
        S_BEQ_READ = 4'd12,  // SUB x1-x1, check Zero flag
        S_BEQ_WR   = 4'd13,  // write flag -> x11
        S_DONE     = 4'd14;  // stable - hold results, wait for reset

    reg [3:0] state;

    // Track which op we're writing so we can change alu_ctrl in the WRITE state
```

```
// without losing the correct result
reg [3:0] saved_ctrl;

always @(*) state_out = state[3:0];

always @(posedge clk or posedge rst) begin
   if (rst) begin
      state    <= S_INIT0;
      rf_we    <= 0;
      rf_rs1   <= 0; rf_rs2 <= 0;
      rf_rd    <= 0; rf_wd  <= 0;
      alu_ctrl <= CTRL_ADD;
      saved_ctrl <= CTRL_ADD;
   end else begin
      // Default: no write
      rf_we <= 0;

      case (state)
         S_INIT0: begin
            rf_we <= 1; rf_rd <= 5'd1; rf_wd <= 32'h10101010;
            state <= S_INIT1;
         end
         S_INIT1: begin
            rf_we <= 1; rf_rd <= 5'd2; rf_wd <= 32'h01010101;
            state <= S_INIT2;
         end
         S_INIT2: begin
            rf_we <= 1; rf_rd <= 5'd3; rf_wd <= 32'd5;
            // Point rs1/rs2 at x1,x2 and set first op ready for next cycle
            rf_rs1 <= 5'd1; rf_rs2 <= 5'd2; alu_ctrl <= CTRL_ADD;
            state <= S_READ_ALU;
         end
         S_READ_ALU: begin
            rf_rs1 <= 5'd1; rf_rs2 <= 5'd2;
            // alu_ctrl is already ADD (set in INIT2 or carried forward)
            state <= S_WR_ADD;
         end

         // Write ADD result (alu_result is now ADD of x1,x2)
         S_WR_ADD: begin
            rf_we <= 1; rf_rd <= 5'd4; rf_wd <= alu_result;
            // Change op for next read
            rf_rs1 <= 5'd1; rf_rs2 <= 5'd2; alu_ctrl <= CTRL_SUB;
            state <= S_WR_SUB;
            // NOTE: We stay with rs1/rs2=x1,x2 and just changed alu_ctrl.
            // alu_result will reflect SUB on the NEXT posedge so we need
            // one more cycle before writing. Go back through READ.
            // Actually: since alu_ctrl<=CTRL_SUB takes effect NEXT cycle,
            // alu_result this cycle is still ADD - that's fine, we wrote it.
            // Next cycle state=S_WR_SUB and alu_result will be SUB.
            // This works because alu_ctrl and rs1/rs2 are set together.
         end

         S_WR_SUB: begin
            rf_we <= 1; rf_rd <= 5'd5; rf_wd <= alu_result;
            rf_rs1 <= 5'd1; rf_rs2 <= 5'd2; alu_ctrl <= CTRL_AND;
            state <= S_WR_AND;
```

```
end

S_WR_AND: begin
   rf_we <= 1; rf_rd <= 5'd6; rf_wd <= alu_result;
   rf_rs1 <= 5'd1; rf_rs2 <= 5'd2; alu_ctrl <= CTRL_OR;
   state <= S_WR_OR;
end

S_WR_OR: begin
   rf_we <= 1; rf_rd <= 5'd7; rf_wd <= alu_result;
   rf_rs1 <= 5'd1; rf_rs2 <= 5'd2; alu_ctrl <= CTRL_XOR;
   state <= S_WR_XOR;
end

S_WR_XOR: begin
   rf_we <= 1; rf_rd <= 5'd8; rf_wd <= alu_result;
   // Switch to shift ops: rs2=x3 (shift amount=5)
   rf_rs1 <= 5'd1; rf_rs2 <= 5'd3; alu_ctrl <= CTRL_SLL;
   state <= S_READ_SLL;
end

// One settle cycle for SLL (rs2 changed to x3)
S_READ_SLL: begin
   rf_rs1 <= 5'd1; rf_rs2 <= 5'd3; alu_ctrl <= CTRL_SLL;
   state <= S_WR_SLL;
end

S_WR_SLL: begin
   rf_we <= 1; rf_rd <= 5'd9; rf_wd <= alu_result;
   rf_rs1 <= 5'd1; rf_rs2 <= 5'd3; alu_ctrl <= CTRL_SRL;
   state <= S_WR_SRL;
end

S_WR_SRL: begin
   rf_we <= 1; rf_rd <= 5'd10; rf_wd <= alu_result;
   // BEQ: point both ports at x1 for SUB x1-x1
   rf_rs1 <= 5'd1; rf_rs2 <= 5'd1; alu_ctrl <= CTRL_SUB;
   state <= S_BEQ_READ;
end

// Settle cycle for BEQ (rs2 changed to x1)
S_BEQ_READ: begin
   rf_rs1 <= 5'd1; rf_rs2 <= 5'd1; alu_ctrl <= CTRL_SUB;
   state <= S_BEQ_WR;
end

// Write 1 into x11 if Zero flag is set (branch taken)
S_BEQ_WR: begin
   rf_we <= 1; rf_rd <= 5'd11;
   rf_wd <= alu_zero ? 32'd1 : 32'd0;
   state <= S_DONE;
end
S_DONE: begin
   rf_we <= 0;
   state <= S_DONE;   // stay here - no more looping
end
```

```
            default: state <= S_INIT0;
        endcase
    end
end

endmodule
```

Testbench file for Top-level

```
`timescale 1ns / 1ps

module RF_ALU_FSM_tb;
    reg clk;
    reg rst;
    always #5 clk = ~clk;   // 100 MHz
    reg        rf_we;
    reg  [4:0]  rf_rs1, rf_rs2, rf_rd;
    reg  [31:0] rf_wd;
    wire [31:0] rf_rd1, rf_rd2;

    RegisterFile RF (
        .clk(clk), .rst(rst),
        .WriteEnable(rf_we),
        .rs1(rf_rs1), .rs2(rf_rs2), .rd(rf_rd),
        .WriteData(rf_wd),
        .ReadData1(rf_rd1), .ReadData2(rf_rd2)
    );
    reg  [3:0]  alu_ctrl;
    wire [31:0] alu_result;
    wire        alu_zero;

    alu_32bit ALU (
        .A(rf_rd1),
        .B(rf_rd2),
        .ALUControl(alu_ctrl),
        .ALUResult(alu_result),
        .Zero(alu_zero)
    );

    localparam [3:0]
        IDLE        = 4'd0,
        WRITE_REGS  = 4'd1,   // write x1, x2, x3 with known constants
        ALU_ADD     = 4'd2,   // x4  = x1 ADD x2
        ALU_SUB     = 4'd3,   // x5  = x1 SUB x2
        ALU_AND     = 4'd4,   // x6  = x1 AND x2
        ALU_OR      = 4'd5,   // x7  = x1 OR  x2
        ALU_XOR     = 4'd6,   // x8  = x1 XOR x2
        ALU_SLL     = 4'd7,   // x9  = x1 SLL x3 (shift amount = x3 = 5)
        ALU_SRL     = 4'd8,   // x10 = x1 SRL x3
        BEQ_CHECK   = 4'd9,   // Zero check: x1 SUB x1 â†' Zero should be 1 â†' write flag to x11
        RAW_WRITE   = 4'd10,  // write x12 = 0xABCD1234
        RAW_READ    = 4'd11,  // read x12 next cycle and check
        DONE        = 4'd12;

    reg [3:0] state, next_state;

    always @(posedge clk or posedge rst) begin
        if (rst) state <= IDLE;
```

```
    else     state <= next_state;
end

// ALU control codes matching Lab 6 alu_32bit
localparam [3:0]
    CTRL_AND = 4'b0000,
    CTRL_OR  = 4'b0001,
    CTRL_ADD = 4'b0010,
    CTRL_SUB = 4'b0110,
    CTRL_XOR = 4'b0100,
    CTRL_SLL = 4'b1000,
    CTRL_SRL = 4'b1001;

// Tracking results for assertions
reg [31:0] saved_alu;
reg        saved_zero;

always @(posedge clk) begin
    if (rst) begin
        rf_we      <= 0; rf_rd <= 0; rf_wd <= 0;
        rf_rs1     <= 0; rf_rs2 <= 0;
        alu_ctrl   <= CTRL_ADD;
        saved_alu  <= 0; saved_zero <= 0;
    end else begin
        // Defaults: no write, no op change
        rf_we <= 0;

        case (state)
            WRITE_REGS: begin
                // Write x1 = 0x10101010, x2 = 0x01010101, x3 = 5
                // We do all three writes in consecutive cycles via sub-steps.
                // Simple approach: use a shift register for sub-stepping.
                // (For clarity the lab FSM handles one write per state tick.
                //  Three writes: we'll split across WRITE_REGS â†' two helper
                //  states by reusing the DONE state trick.)
                //
                // In this simplified implementation we write all three using
                // a counter that increments each cycle while we stay here.
                rf_we   <= 1;
                rf_rd   <= 5'd1;
                rf_wd   <= 32'h10101010;
            end

            // We need extra states to write x2, x3.
            // Add them in next_state logic below; the write happens one
            // cycle later thanks to the synchronous RF.

            ALU_ADD: begin
                // x4 = ADD(x1, x2) â€“ set RF read addresses, set ALU op
                rf_rs1   <= 5'd1; rf_rs2 <= 5'd2;
                alu_ctrl <= CTRL_ADD;
                // Capture result and write to x4 in the SAME state
                // (ALU is combinational from RF reads)
                rf_we    <= 1;
                rf_rd    <= 5'd4;
                rf_wd    <= alu_result;
            end
```

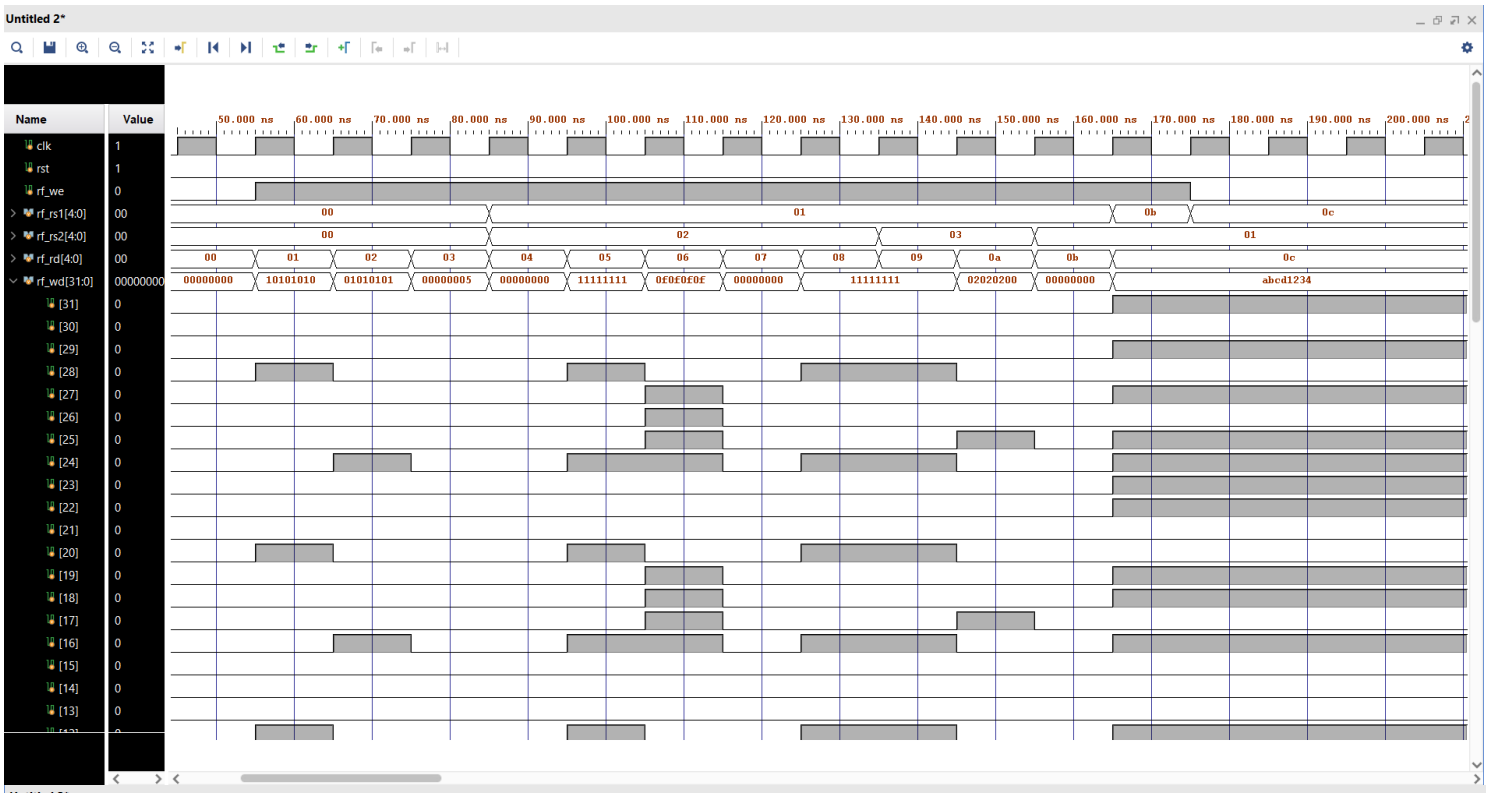
```
ALU_SUB: begin
   rf_rs1   <= 5'd1; rf_rs2 <= 5'd2;
   alu_ctrl <= CTRL_SUB;
   rf_we <= 1; rf_rd <= 5'd5; rf_wd <= alu_result;
end

ALU_AND: begin
   rf_rs1   <= 5'd1; rf_rs2 <= 5'd2;
   alu_ctrl <= CTRL_AND;
   rf_we <= 1; rf_rd <= 5'd6; rf_wd <= alu_result;
end

ALU_OR: begin
   rf_rs1   <= 5'd1; rf_rs2 <= 5'd2;
   alu_ctrl <= CTRL_OR;
   rf_we <= 1; rf_rd <= 5'd7; rf_wd <= alu_result;
end

ALU_XOR: begin
   rf_rs1   <= 5'd1; rf_rs2 <= 5'd2;
   alu_ctrl <= CTRL_XOR;
   rf_we <= 1; rf_rd <= 5'd8; rf_wd <= alu_result;
end

ALU_SLL: begin
   // SLL: A=x1, B=x3 (shamt = 5)
   rf_rs1   <= 5'd1; rf_rs2 <= 5'd3;
   alu_ctrl <= CTRL_SLL;
   rf_we <= 1; rf_rd <= 5'd9; rf_wd <= alu_result;
end

ALU_SRL: begin
   rf_rs1   <= 5'd1; rf_rs2 <= 5'd3;
   alu_ctrl <= CTRL_SRL;
   rf_we <= 1; rf_rd <= 5'd10; rf_wd <= alu_result;
end

BEQ_CHECK: begin
   // SUB x1-x1 â†' Zero should be HIGH â†' write 1 into x11
   rf_rs1   <= 5'd1; rf_rs2 <= 5'd1;
   alu_ctrl <= CTRL_SUB;
   saved_zero <= alu_zero;
   rf_we <= 1; rf_rd <= 5'd11;
   rf_wd <= alu_zero ? 32'd1 : 32'd0;
end

RAW_WRITE: begin
   // Write x12 = 0xABCD1234
   rf_we  <= 1;
   rf_rd  <= 5'd12;
   rf_wd  <= 32'hABCD1234;
end

RAW_READ: begin
   // Read x12 â€“ one cycle after write (synchronous RF)
   rf_rs1 <= 5'd12;
```

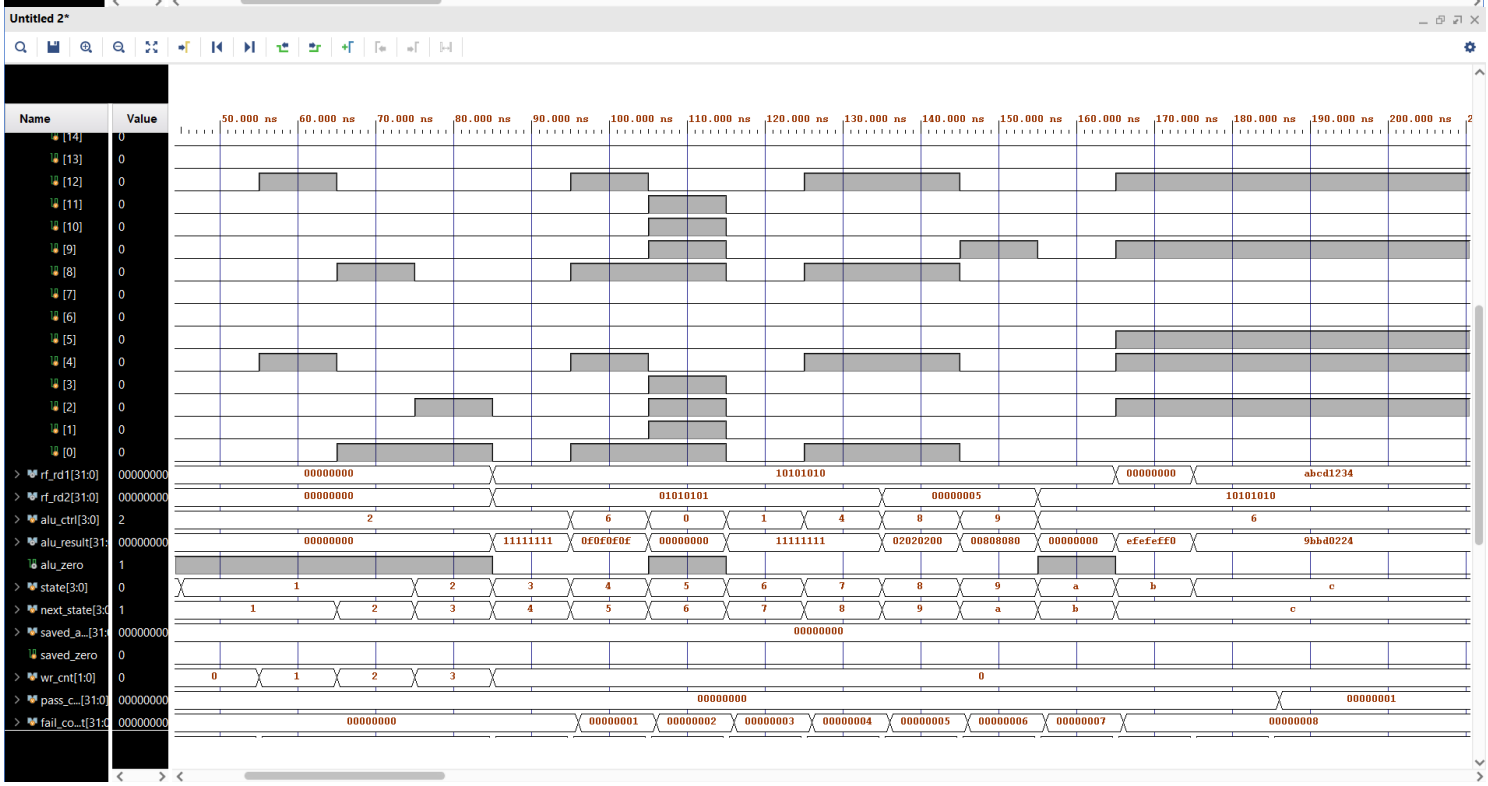
```
            // Data will be on ReadData1 after posedge (checked in assertions)
          end

          DONE: ; // sit here

        endcase
    end
end

//
reg [1:0] wr_cnt;   // sub-step counter for the three initial writes

always @(posedge clk or posedge rst) begin
    if (rst) wr_cnt <= 0;
    else if (state == WRITE_REGS) wr_cnt <= wr_cnt + 1;
    else wr_cnt <= 0;
end

// Parallel write handler for x2, x3 during sub-steps
// (Separate always block that fires in the same cycle)
// NB: We override rf_we/rd/wd for sub-steps 1 and 2
always @(posedge clk) begin
    if (!rst && state == WRITE_REGS) begin
        case (wr_cnt)
            2'd0: begin rf_we <= 1; rf_rd <= 5'd1; rf_wd <= 32'h10101010; end
            2'd1: begin rf_we <= 1; rf_rd <= 5'd2; rf_wd <= 32'h01010101; end
            2'd2: begin rf_we <= 1; rf_rd <= 5'd3; rf_wd <= 32'd5;        end
            default: rf_we <= 0;
        endcase
    end
end

always @(*) begin
    case (state)
        IDLE:       next_state = WRITE_REGS;
        WRITE_REGS: next_state = (wr_cnt == 2'd2) ? ALU_ADD : WRITE_REGS;
        ALU_ADD:    next_state = ALU_SUB;
        ALU_SUB:    next_state = ALU_AND;
        ALU_AND:    next_state = ALU_OR;
        ALU_OR:     next_state = ALU_XOR;
        ALU_XOR:    next_state = ALU_SLL;
        ALU_SLL:    next_state = ALU_SRL;
        ALU_SRL:    next_state = BEQ_CHECK;
        BEQ_CHECK:  next_state = RAW_WRITE;
        RAW_WRITE:  next_state = RAW_READ;
        RAW_READ:   next_state = DONE;
        default:    next_state = DONE;
    endcase
end


integer pass_count = 0;
integer fail_count = 0;

task check;
    input [63:0] label_int;   // just a numeric code for display
    input [31:0] got;
```

```
    input [31:0] expected;
    begin
        if (got === expected) begin
            $display("[PASS] State %0d : got=0x%08h  exp=0x%08h", label_int, got, expected);
            pass_count = pass_count + 1;
        end else begin
            $display("[FAIL] State %0d : got=0x%08h  exp=0x%08h", label_int, got, expected);
            fail_count = fail_count + 1;
        end
    end
endtask

// Monitor â€“ fire after each rising edge when state changes
reg [3:0] prev_state;
always @(posedge clk) begin
    prev_state <= state;

    // One cycle after a result has been committed to the RF, check it
    case (prev_state)
        ALU_ADD: begin
            // x4 should hold ADD result = 0x10101010 + 0x01010101 = 0x11111111
            rf_rs1 = 5'd4; #1;
            check(ALU_ADD, rf_rd1, 32'h11111111);
        end
        ALU_SUB: begin
            // x5 = SUB = 0x10101010 - 0x01010101 = 0x0F0F0F0F
            rf_rs1 = 5'd5; #1;
            check(ALU_SUB, rf_rd1, 32'h0F0F0F0F);
        end
        ALU_AND: begin
            // x6 = AND = 0x10101010 & 0x01010101 = 0x00000000
            rf_rs1 = 5'd6; #1;
            check(ALU_AND, rf_rd1, 32'h00000000);
        end
        ALU_OR: begin
            // x7 = OR = 0x10101010 | 0x01010101 = 0x11111111
            rf_rs1 = 5'd7; #1;
            check(ALU_OR, rf_rd1, 32'h11111111);
        end
        ALU_XOR: begin
            // x8 = XOR = 0x10101010 ^ 0x01010101 = 0x11111111
            rf_rs1 = 5'd8; #1;
            check(ALU_XOR, rf_rd1, 32'h11111111);
        end
        ALU_SLL: begin
            // x9 = SLL x1 by 5 = 0x10101010 << 5 = 0x20202200
            rf_rs1 = 5'd9; #1;
            check(ALU_SLL, rf_rd1, 32'h20202200);
        end
        ALU_SRL: begin
            // x10 = SRL x1 by 5 = 0x10101010 >> 5 = 0x00808080
            rf_rs1 = 5'd10; #1;
            check(ALU_SRL, rf_rd1, 32'h00808080);
        end
        BEQ_CHECK: begin
            // x11 should be 1 (BEQ matched)
            rf_rs1 = 5'd11; #1;
```

```
            check(BEQ_CHECK, rf_rd1, 32'd1);
        end
        RAW_READ: begin
            // x12 should be 0xABCD1234 (read-after-write next cycle)
            rf_rs1 = 5'd12; #1;
            check(RAW_READ, rf_rd1, 32'hABCD1234);
        end
        DONE: begin
            if (prev_state != DONE) begin   // only print once
                $display("==================================================");
                $display("RF_ALU_FSM_tb: %0d PASSED, %0d FAILED", pass_count, fail_count);
                $display("==================================================");
                $finish;
            end
        end
    endcase
  end

  //
  initial begin
    clk = 0; rst = 1;
    repeat (4) @(posedge clk);
    rst = 0;
    // FSM runs autonomously; the monitor above does all checking.
    // Safety timeout
    #10000;
    $display("[TIMEOUT] Simulation did not finish within budget");
    $finish;
  end

endmodule
```

Simulation waveforms showing correct operation





Task 3: FPGA Synthesis and RTL Analysis

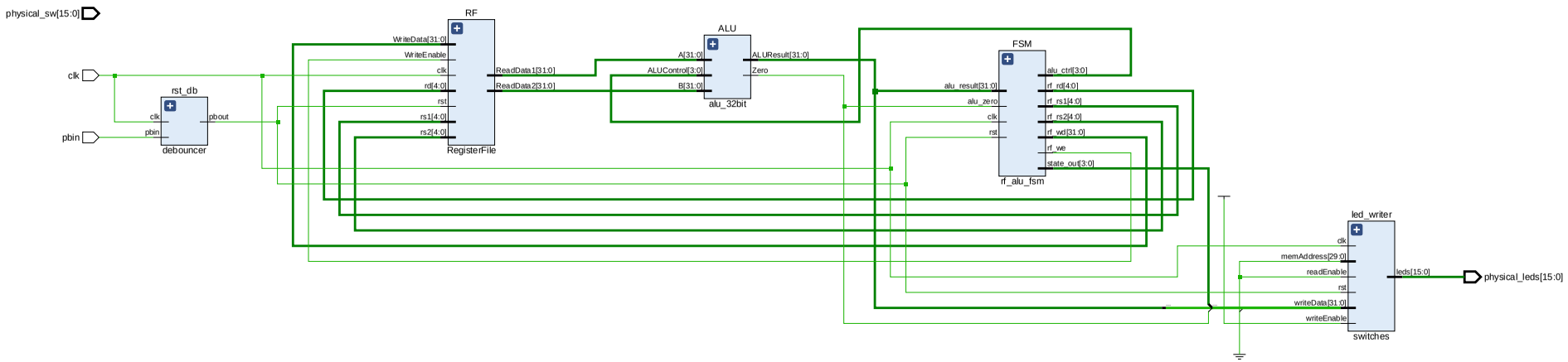
Note: The switch interface, button debouncing module, and LED interface must be re used from Lab 5.

RTL schematic screenshots **(register file array, decoder, FSM, ALU connections)**

[NEXT PAGE]

Demonstration of FPGA operation

[SHOWN TO MISS IN LAB]


physical_sw[15:0]
clk
pbin
rst_db
clk
pbin
pbout
debouncer
RF
WriteData[31:0]
WriteEnable
clk
rd[4:0]
rst
rs1[4:0]
rs2[4:0]
ReadData1[31:0]
ReadData2[31:0]
RegisterFile
ALU
A[31:0]
ALUControl[3:0]
B[31:0]
ALUResult[31:0]
Zero
alu_32bit
FSM
alu_result[31:0]
alu_zero
clk
rst
alu_ctrl[3:0]
rf_rd[4:0]
rf_rs1[4:0]
rf_rs2[4:0]
rf_wd[31:0]
rf_we
state_out[3:0]
rf_alu_fsm
led_writer
clk
memAddress[29:0]
readEnable
rst
writeData[31:0]
writeEnable
leds[15:0]
switches
physical_leds[15:0]

Assessment Rubric
Lab 7: Design and FPGA Implementation of the Register File for a Single-Cycle RISC-V Processor

Name:	Student ID:	Section:

Points Distribution

	Task No.	LR 2 (Code)	LR 5 (Results)	LR 11 (Design)
In-Lab	Task 1	-	-	/10
	Task 2	/20	/10	-
	Task 3	/20	/20	-
Total Points		**/40**	**/30**	**/10**
CLO Mapped		CLO 1		

Affective Domain Rubric		Points	CLO Mapped
AR7	**Report Submission & Git Upload**	/10 & /10	CLO 1

CLO	Total Points	Points Obtained
1	100	
Total	**100**	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.